DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Ejercicios de modelado, simulación y realización de circuitos combinacionales**	**CÓDIGO:** **ANP1**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Libre. Ejercicios	TIPO:	No Presencial	DURACIÓN:	360 minutos

CALENDARIO:	PS4, PS5 y PS6	REQUISITOS:	Fundamentos de modelado VHDL

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Descripción de la actividad

La realización de los ejercicios que componen esta actividad persigue diversos objetivos. El primero de ellos es practicar el desarrollo de modelos lógicos sintetizables VHDL de subsistemas combinacionales simples y de los test-benches necesarios para verificarlos, tareas que pretenden facilitarle la asimilación de las reglas para la correcta construcción de los modelos y test-benches y que se vaya familiarizando con las construcciones sintácticas básicas del lenguaje; la realización de los ejercicios obliga, además, a un uso intensivo del simulador ModelSim que le permitirá practicar y asentar los conocimientos adquiridos en los tutoriales de manejo del simulador que ya ha realizado. Los ejercicios le servirán también para repasar las características de diversos subsistemas combinacionales de uso común: de hecho, los ejercicios están clasificados en apartados que atienden a la ordenación funcional que clásicamente se hace de dichos subsistemas (multiplexores, decodificadores, codificadores, etcétera), estando prologados por una breve introducción que le recuerda las características básicas de cada tipo de subsistema y una propuesta de cómo abordar la codificación de su modelo lógico en VHDL.

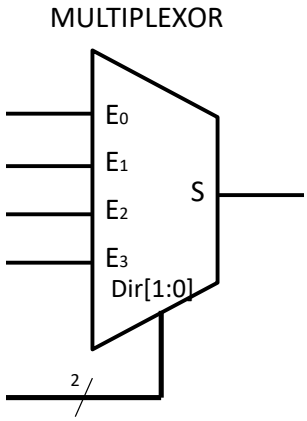
Cuando comience a realizar estos ejercicios es probable que tenga que consultar la documentación correspondiente a los tutoriales de manejo del simulador ModelSim y las presentaciones en las que se describe cómo realizar modelos y test-benches de circuitos combinacionales simples. A medida que vaya realizando ejercicios y alcanzando los objetivos de la actividad, las consultas deberían cesar.

Esta actividad puede realizarla en la modalidad que desee (individualmente, en pareja o con un grupo de compañeros más numeroso) y también puede elegir los ejercicios concretos que va a realizar; tome como referencia los objetivos de aprendizaje mencionados en el primer párrafo de esta introducción y, en base al grado de adquisición de los mismos, decida autónomamente cuando debe darla por finalizada.

Queda por decir, para terminar, que los ejercicios de esta actividad proponen que realice únicamente tareas de modelado y verificación. Pero resulta conveniente que realice alguna práctica de realización del circuito sobre la tarjeta MAX1000. Para ello elija, de entre los ejercicios que haya realizado, un par de ellos que sean susceptibles de ser probados en dicha tarjeta y utilícelos para practicar el flujo de diseño con Quartus Prime y el manejo de la tarjeta de prototipado.

1.- Multiplexores

Un multiplexor es un subsistema combinacional que dispone de *n* entradas de datos de 1 bit, una entrada de control de selección de *m* bits (tal que $2^m \geq n$) y una salida de un bit; el nivel lógico de la salida es el correspondiente a la entrada direccionada mediante la entrada de selección. El símbolo de la figura representa la interfaz de un multiplexor con cuatro canales de entrada (cuatro entradas de datos).



El funcionamiento de este multiplexor puede modelarse fácilmente empleando una estructura *if-elsif* o una sentencia *case*:

```
library ieee;
use ieee.std_logic_1164.all;

entity mux_4canales is
port(E:   in     std_logic_vector(3 downto 0);
     Dir: in     std_logic_vector(1 downto 0);
     S:   buffer std_logic);

end entity;
```

```
architecture rtl_1 of mux_4canales is
begin
  process(E, Dir)
  begin
    if Dir = "00" then
      S <= E(0);

    elsif Dir = "01" then
      S <= E(1);

    elsif Dir = "10" then
      S <= E(2);

    elsif Dir = "11" then
      S <= E(3);

    else
      S <= 'X';

    end if;
  end process;
end rtl_1;
```

```
architecture rtl_2 of mux_4canales is
begin
  process(E, Dir)
  begin
    case Dir is
      when "00" =>
        S <= E(0);

      when "01" =>
        S <= E(1);

      when "10" =>
        S <= E(2);

      when "11" =>
        S <= E(3);

      when others =>
        S <= 'X';

    end case;
  end process;
end rtl_2;
```

El modelo podría verificarse comprobando la selección de cada canal de entrada para cada nivel lógico:

- Se selecciona el canal 0 (Dir = “00”) cuando tiene un 0 (E = “1110”)
- Se selecciona el canal 1 (Dir = “01”) cuando tiene un 0 (E = “1101”)
- Se selecciona el canal 2 (Dir = “10”) cuando tiene un 0 (E = “1011”)
- Se selecciona el canal 3 (Dir = “11”) cuando tiene un 0 (E = “0111”)
- Se selecciona el canal 0 (Dir = “00”) cuando tiene un 1 (E = “0001”)
- Se selecciona el canal 1 (Dir = “01”) cuando tiene un 1 (E = “0010”)
- Se selecciona el canal 2 (Dir = “10”) cuando tiene un 1 (E = “0100”)
- Se selecciona el canal 3 (Dir = “11”) cuando tiene un 1 (E = “1000”)

Esta prueba, en la que se cubren sólo 8 de las 64 posibles combinaciones de entrada, resulta suficiente para dar por válido el modelo y puede codificarse, por ejemplo, tal y como se muestra en el siguiente proceso:

```
process
begin
  Dir <= "00";
  E   <= "1110";
  wait for 100 ns;            -- Cada combinación 100 ns

  Dir <= "01";
  E   <= E(2 downto 0)&E(3); -- "1101" <-> "110" & '1'
  wait for 100 ns;

  Dir <= "10";
  E   <= E(2 downto 0)&E(3); -- "1011" <-> "101" & '1'
  wait for 100 ns;

  Dir <= "11";
  E   <= E(2 downto 0)&E(3); -- "0111" <-> "011" & '1'
  wait for 100 ns;

  E <= not E;                 -- "1000" <-> not "0111"
  wait for 100 ns;

  Dir <= "10";
  E   <= E(0)&E(3 downto 1); -- "0100" <-> '0' & "100"
  wait for 100 ns;

  Dir <= "01";
  E   <= E(0)&E(3 downto 1); -- "0010" <-> '0' & "010"
  wait for 100 ns;

  Dir <= "00";
  E   <= E(0)&E(3 downto 1); -- "0001" <-> '0' & "001"
  wait for 100 ns;

  wait;                       -- Fin de la simulación
end process;
```

Observe que este código muestra cómo puede emplearse el operador de encadenamiento, ‘&’, para desplazar los bits de un dato hacia la izquierda o la derecha.

Ejercicio 1.1

Edite y compile una de las versiones del modelo del multiplexor de 4 canales. Realice un *test-bench* para verificar el modelo del multiplexor (empleando *exactamente* el código del proceso anterior) y ejecute una simulación para comprobar su corrección.

Ejercicio Avanzado[1] 1.2

Es bastante frecuente que los diseñadores con poca experiencia en el empleo de HDLs olviden incluir alguna de las entradas de un circuito combinacional en la lista de sensibilidad de los procesos que modelan su funcionamiento –especialmente cuando se trata de una entrada de *control* como, por ejemplo, la entrada de selección de un multiplexor. Este hecho puede pasar, o no, inadvertido al revisar los resultados de la simulación y tiene especial gravedad pues da lugar al modelado de un *latch*, un circuito con memoria (un combinacional realimentado), que si se traslada al hardware ocasionará con toda seguridad problemas de funcionamiento.

Modifique el modelo del multiplexor del ejercicio 1.1, eliminando de la lista de sensibilidad del proceso la entrada de selección del multiplexor. Recompile el modelo y el *test-bench*, ejecute una simulación y, finalmente, revise los resultados para contestar las siguientes cuestiones.

a) Indique, aquí, en qué intervalos de la simulación se observa un comportamiento incorrecto y en cuáles no.

b) Determine las causas por las que el modelo, evidentemente incorrecto, muestra en todas las ocasiones una respuesta correcta.

c) Teniendo en cuenta únicamente los resultados de la simulación: ¿Puede deducirse de los mismos que el circuito (incorrectamente) modelado tiene memoria? ¿Por qué no[2]? Diseñe (o esboce las condiciones-de) una prueba en la que sí pudiera comprobarse esta propiedad del incorrecto modelo.

[1] El aprendizaje que podría derivarse de la realización de este ejercicio es muy importante (e interesante), pero si considera que su nivel de destreza es aún muy bajo, es mejor que se centre en el repaso y en los ejercicios de modelado y simulación.
[2] Ayudita.

Nota: La realización de los siguientes ejercicios puede acelerarse **muchísimo** si reaprovecha el código (correcto) del ejercicio 1.1 y de los que vaya posteriormente realizando.

Ejercicio 1.3

Realice un modelo VHDL sintetizable de un multiplexor de 8 canales. Diseñe un conjunto de pruebas que permitan verificar el modelo para, a continuación, realizar un *test-bench* y ejecutar una simulación que le permita validarlo.

Ejercicio 1.4

Un selector de datos es un multiplexor en el que cada canal de entrada tiene varios bits; por tanto, cuando se selecciona un determinado canal se transfiere a la salida el dato presente en el mismo. Realice un modelo VHDL sintetizable de un selector de datos con 10^3 canales de 8 bits cada uno. Diseñe un conjunto de pruebas que permitan verificar el modelo para, a continuación, realizar un *test-bench* y ejecutar una simulación que le permitan validarlo.

Ejercicio 1.5

Realice el modelo VHDL de un selector de datos como el del ejercicio anterior, pero que cuente con:

- una entrada de habilitación (*ena*) que, cuando esté desactivada (a nivel bajo), mantenga a nivel bajo todos los bits de la salida del selector

- una entrada de control *(inv*) que, cuando esté activa (a nivel alto) invierta en la salida los bits del canal de entrada seleccionado; la actividad de la entrada *ena* tiene prioridad sobre la de *inv*.

Modifique el diseño de las pruebas para verificar, también, la nueva funcionalidad; realice el *test-bench* y ejecute una simulación que le permita validar el modelo.

Ejercicio Avanzado 1.6

Considere que tiene la necesidad de realizar un selector de datos que permita seleccionar uno de entre 1000 datos de 64 bits. Teniendo en cuenta la experiencia y conocimientos de los que ya dispone en el modelado de multiplexores:

- ¿Le parece razonable extrapolar la técnica que ha empleado en los ejercicios anteriores para resolver el problema que presenta la construcción del modelo de este multiplexor gigante? ¿Sí? ¿No? ¿Por qué?

[3] No es una errata, son 10.

2.- Decodificadores

En el área del diseño de circuitos digitales se denomina decodificación al proceso de *detección* de la ocurrencia de una, o de una de entre un grupo, de las palabras que forman parte de un determinado código; un decodificador es un circuito combinacional que realiza una decodificación.

La definición anterior engloba a diversos tipos de circuitos. De entre todos ellos, los más interesantes, por ser los de uso más común, son aquellos en los que el código a *decodificar* representa números binarios: los *decodificadores binarios*.

Un decodificador binario cuenta con m bits de entrada y n bits de salida, con $n = 2^m$. La entrada codifica números en binario natural y la salida indica la detección de cada número empleando un código *one-hot*[4]; cuando en la entrada se ingresa la combinación correspondiente al número i, se activa, únicamente, la salida i de entre las n disponibles –en un decodificador binario con tres bits de entrada, por ejemplo, si la combinación de entrada es 111 (7), se activa únicamente la salida 7 (S_7), el resto permanecen inactivas (10000000); si en la entrada hay un 4 (100), se activa la salida 4 (S_4, 00010000), etcétera.

Otro ejemplo de decodificador de números es el decodificador decimal (o BCD). Este decodificador tiene 4 bits de entrada que codifican un número BCD y 10 bits de salida; en este tipo de decodificadores se activa, en cada momento, la salida (de la S_0 a la S_9) correspondiente al número BCD que haya en la entrada.

Un decodificador BCD a 7 segmentos decodifica números decimales y emplea un código de salida que no es *one-hot* ya que está diseñado para dibujar el número decodificado en un *display* de 7 segmentos.

El siguiente código corresponde a un modelo VHDL sintetizable de un decodificador binario de números de 2 bits.

```
entity decod_2bits is
port(E: in     std_logic_vector(1 downto 0);
     S: buffer std_logic_vector(3 downto 0));

end entity;
```

```
architecture rtl of decod_2bits is
begin
  process(E)
  begin
    case E is
      when "00" =>
        S <= "0001";

      when "01" =>
        S <= "0010";

      when "10" =>
        S <= "0100";

      when "11" =>
        S <= "1000";

      when others =>
        S <= "XXXX";

    end case;
  end process;
end rtl;
```

[4] Un código *one-hot* de n bits es aquel en el que en cada palabra del código sólo hay un bit activo –las palabras de un código *one-hot* de 3 bits son: 001, 010 y 100. Un código *one-hot* de n bits cuenta, por tanto, con n palabras diferentes.

Ejercicio 2.1

Edite y compile el modelo VHDL del decodificador binario de 2 bits. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Nota: La realización de los siguientes ejercicios puede acelerarse si reaprovecha el código del ejercicio 2.1 y de los que vaya posteriormente realizando.

Ejercicio 2.2

Realice el modelo VHDL sintetizable de un decodificador binario para números de 3 bits con entradas y salidas activas a nivel bajo. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 2.3

Realice el modelo VHDL sintetizable de un decodificador como el del ejercicio 2.2 pero que cuente con una entrada de habilitación activa a nivel alto. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 2.4

En ocasiones es necesario detectar la ocurrencia de un determinado valor númerico en el código de entrada (en este caso la decodificación coincide con la comparación del código de entrada con un número). Realice el modelo VHDL sintetizable de un circuito que decodifique el número 98 en una palabra de 8 bits. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

¿Qué cambios tendría que realizar en el modelo si el código de entrada empleado fuera complemento a 2 y hubiera que detectar la ocurrencia del número -76?

Los siguientes ejercicios corresponden a circuitos decodificadores en los que el código de entrada no es numérico.

Ejercicio 2.5

Realice el modelo VHDL de un circuito que detecte las palabras del código de entrada, de 4 bits, que contienen exactamente 3 unos. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 2.6

Realice el modelo VHDL de un circuito que detecte las palabras del código ASCII de entrada que corresponden a letras minúsculas. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

3.- Codificadores

Un codificador es un circuito que genera diferentes palabras de código dependiendo de las entradas que tenga activas. Los codificadores que se utilizan más frecuentemente son los codificadores binarios y decimales.

Un codificador binario con n entradas y m salidas, donde $n = 2^m$, indica en su salida, en binario natural, el número de la entrada que está activa –si, por ejemplo, es un codificador con 3 bits de salida y está activa la entrada 6 (E_6), en la salida se entrega el código 110.

Si en la entrada de un codificador hay más de una entrada activa, el circuito puede responder de dos maneras distintas:

- Como un *codificador con prioridad*, en cuyo caso entregará en la salida el código correspondiente al número de la entrada más prioritaria

- Como un *codificador sin prioridad*, en cuyo caso entregará un código que, en general, no se corresponderá con ninguna de las entradas activas; obsérvese que este tipo de codificadores sólo pueden emplearse si existen garantías de que nunca va a tener que realizarse una codificación con más de una entrada activa

El siguiente código corresponde a un codificador *sin prioridad* de 3 bits:

```
library ieee;
use ieee.std_logic_1164.all;

entity cod_3bits_mafr is
port(E: in     std_logic_vector(7 downto 0);
     S: buffer std_logic_vector(2 downto 0));

end entity;

architecture rtl of cod_3bits_mafr is
begin
  process(E)
  begin
    case E is
      when "00000000"|"00000001" => S <= "000";
      when "00000010" =>            S <= "001";
      when "00000100" =>            S <= "010";
      when "00001000" =>            S <= "011";
      when "00010000" =>            S <= "100";
      when "00100000" =>            S <= "101";
      when "01000000" =>            S <= "110";
      when "10000000" =>            S <= "111";
      when others =>                S <= "XXX";

    end case;
  end process;
end rtl;
```

Observe que la combinación 000 se establece bien por la activación de E_0, o bien porque ninguna de las entradas está activa. Si es necesario distinguir entre ambos estados de entrada, hay que añadir un *testigo* en la salida que se active únicamente cuando alguno de los bits de entrada esté a 1.

Ejercicio 3.1

Edite y compile el modelo VHDL del codificador binario de 3 bits. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Nota: La realización de los siguientes ejercicios puede acelerarse si reaprovecha el código del ejercicio 3.1 y de los que vaya posteriormente realizando.

Ejercicio 3.2

Realice el modelo VHDL sintetizable de un *codificador binario con prioridad* de 2 bits con entradas y salidas activas a nivel bajo. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 3.3

Realice el modelo VHDL sintetizable de un codificador como el del ejercicio 3.2 pero que cuente con una salida de validación de código (*out_val*) que se active cuando cualquiera de las entradas del codificador esté también activa. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 3.4

Realice el modelo VHDL sintetizable de un codificador BCD, con entradas y salidas activas a nivel alto y que cuente con una salida de validación de código (*out_val*). Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

4.- Detectores de Paridad

Un circuito detector de paridad es un circuito que activa su salida, de 1 bit, cuando el número de unos que hay en el dato de entrada, de *n* bits, cumple con el criterio de paridad (número *par* o *impar* de unos).

Si el detector emplea el criterio de paridad impar, la salida del detector puede calcularse como la función *or-exclusiva* de todos los bits del dato de entrada. Si se emplea el criterio de paridad par, la salida se calcula como la función *or-exclusiva* negada de todos los bits de entrada. El siguiente código muestra un modelo VHDL sintetizable de un circuito que genera ambos bits de paridad.

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity paridad_4bits is
port(E:             in     std_logic_vector(3 downto 0);
     paridad_par:   buffer std_logic;
     paridad_impar: buffer std_logic);

end entity;
```

```vhdl
architecture rtl of paridad_4bits is
begin
  process(E)
  begin
    paridad_impar <= E(3) xor E(2) xor E(1) xor E(0);
    paridad_par   <= E(3) xnor E(2) xnor E(1) xnor E(0);

  end process;
end rtl;
```

Ejercicio 4.1

Edite y compile el modelo VHDL del detector de paridad de 4 bits con dos salidas. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Nota: La realización de los siguientes ejercicios puede acelerarse si reaprovecha el código del ejercicio 4.1.

Ejercicio 4.2

Edite y compile el modelo VHDL de un detector de paridad de 3 bits con dos salidas. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Nota: Recuerde que la función **xor** de *n* variables es complementaria de la **xnor** únicamente cuando *n* es par; cuando *n* es impar ambas son la misma función.

Ejercicio 4.3

Edite y compile el modelo VHDL de un detector de paridad de 4 bits con una única salida y una entrada de control, *par_impar*, que, a nivel bajo, hace que el circuito detecte paridad par y a nivel alto, paridad impar. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

5.- Generadores de Paridad

Un circuito generador de paridad calcula el valor de un bits de paridad, que es la salida del circuito, de modo que dicho bit junto con los bits de un dato, que es la entrada del circuito, cumplan un criterio de paridad dado (paridad par o impar).

Nota: La realización de los siguientes ejercicios puede acelerarse si reaprovecha el código de los ejercicios del apartado 4.

Ejercicio 5.1

Edite y compile el modelo VHDL de un generador de paridad con dos salidas (bit de paridad par, *bit_par*, y bit de paridad impar, *bit_impar*) par datos de 4 bits. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 5.2

Edite y compile el modelo VHDL de un generador de paridad para datos de 4 bits con una entrada de control, *par_impar*, que, a nivel bajo, hace que el circuito genere un bit para conseguir paridad par y a nivel alto, para conseguir paridad impar. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

6.- Complementador

Un circuito complementador tiene *n* bits de entrada y *n* bits de salida. Cada bit de salida tiene el nivel lógico complementario de su homólogo de entrada.

Ejercicio 6.1

Edite y compile el modelo VHDL de un complementador de datos de 8 bits. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 6.2

Edite y compile el modelo VHDL de un complementador de datos de 8 bits que cuente con una entrada de control, *comp*, que a nivel bajo determina que el circuito entregue en su salida el dato de entrada sin alterar su valor y, a nivel alto, hace que el circuito entregue el dato complementado. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

7.- Ejemplos de funciones combinacionales simples

Los ejercicios siguientes describen funciones combinacionales que pueden modelarse con un único proceso.

Ejercicio 7.1

Realice el modelo VHDL de un circuito que calcula el número de unos que hay en un dato de entrada de 4 bits. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 7.2

Realice el modelo VHDL de un circuito que calcule el valor de la mayor potencia de 2 de la que es múltiplo un número en binario natural de 8 bits. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 7.3

Realice el modelo VHDL de un circuito que determine si el valor absoluto de un número en complemento a 2 de 4 bits es mayor que 5. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 7.4

Realice el modelo VHDL de un circuito que suma 3 a un dígito BCD; la salida del circuito se expresa también en código BCD. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.

Ejercicio 7.5

Realice el modelo VHDL de un circuito que entrega en su salida **S** un número, **E**, de 8 bits, tras haber invertido aquellos cuyo peso es menor al especificado, en binario natural, por una entrada, **INV**, de 4 bits. Diseñe las pruebas para la verificación del modelo, realice un *test-bench* y ejecute una simulación para comprobar su corrección.